

Documentation Technique Complete

Projet NAO — Deplacement autonome par itineraire

Ce document explique en detail absolu le fonctionnement de chaque fichier du projet, import par import, ligne par ligne, ainsi que leurs interactions et la logique globale du systeme de deplacement du robot NAO simule.

Fichier	Langage	Role	Cote
scene_idc.py	Python 3.11	Client — calcul chemin + envoi commandes	PC
serveur_nao.py	Python 2.7	Serveur — reception + deplacement NAO	Choregraphe

1. Architecture generale

Les deux codes forment un systeme **client-serveur**. Le client (scene_idc.py) tourne sur ton PC en Python 3.11. Le serveur (serveur_nao.py) tourne dans une boite Python Script de Choregraphe en Python 2.7. Ils communiquent via **socket TCP** sur le port 9561 en local (adresse 127.0.0.1 = la machine elle-meme).

Pourquoi deux fichiers ? Le NAO est controle via l'API NAOqi qui n'est disponible qu'en Python 2.7 dans Choregraphe. Le client est en Python 3.11 car c'est la version moderne avec toutes les fonctionnalites recentes. La socket TCP permet aux deux versions de Python de communiquer.

scene_idc.py (Python 3.11)		serveur_nao.py (Python 2.7)
PC — Client	<--- TCP 127.0.0.1:9561 --->	Choregraphe — Serveur
Calcule le chemin BFS	---scene JSON---	Recoit la scene
Attend PRET	<---INIT + PRET---	Initialise le NAO
Envoie commandes	---avant/gauche/...---	Execute moveTo()
Attend feedback	<---NAO a avance---	Confirme le mouvement
Verifie case suivante	---verifier:[x,y]---	Consulte la scene
OK ou OBSTACLE	<---OK / OBSTACLE---	Repond selon scene[y][x]
Envoie stop	---stop---	Ferme le serveur

2. Le format JSON — pourquoi et comment

2.1 Qu'est-ce que JSON ?

JSON (JavaScript Object Notation) est un format texte universel pour echanger des donnees entre programmes. Il peut représenter des listes, des dictionnaires, des nombres, des chaines de caracteres. N'importe quel langage sait le lire.

2.2 Pourquoi JSON ici ?

La scene est un tableau Python (liste de listes). Pour l'envoyer via socket, il faut la convertir en texte. JSON est le format ideal car :

- Python 3.11 et Python 2.7 ont tous les deux le module json
- La conversion est reversible sans perte de donnees
- C'est lisible et simple a debugger

2.3 Comment ca fonctionne ligne par ligne

```
# Cote CLIENT (scene_idc.py) :
scene = [['X', 'N'], ['E', '1']]          # tableau Python
scene_json = json.dumps(scene)           # convertit en texte JSON
# scene_json vaut : '[[["X", "N"], ["E", "1"]]]'
client.send((scene_json + "\n").encode())
# encode() convertit le texte Python en bytes (necessaire pour socket)
# + "\n" ajoute un saut de ligne = marqueur de fin de message

# Cote SERVEUR (serveur_nao.py) :
buffer_scene += conn.recv(4096).decode()
# recv(4096) recoit jusqu'a 4096 bytes
# decode() reconvertit les bytes en texte Python
scene_json, reste = buffer_scene.split("\n", 1)
# split("\n", 1) coupe au premier saut de ligne
# scene_json = le JSON de la scene
# reste = ce qui est arrive apres (commandes anticipees)
scene = json.loads(scene_json)
# json.loads() reconvertit le texte JSON en tableau Python
# scene redevient [['X', 'N'], ['E', '1']]
```

Pourquoi le \n ? Une socket TCP est un flux continu de bytes, sans separateur naturel entre les messages. Sans \n, le serveur ne saurait pas ou finit la scene et ou commencent les commandes. Le \n sert de marqueur de fin de message JSON.

3. Tous les imports expliqués

3.1 Imports de scene_idc.py (Python 3.11)

Import	Provenance	Utilisation dans le code
socket	Bibliothèque standard Python	Crée la connexion TCP vers le serveur. <code>socket.socket()</code> , <code>connect()</code> , <code>send()</code> , <code>recv()</code>
threading	Bibliothèque standard Python	Crée des threads parallèles. <code>Thread()</code> , <code>Event()</code> . Permet d'écouter le clavier et le s
os	Bibliothèque standard Python	<code>os.system('cls')</code> efface le terminal Windows pour rafraîchir l'affichage. <code>os._exit(0)</code>
time	Bibliothèque standard Python	<code>time.sleep(1)</code> fait une pause d'1 seconde. Utilise dans le décompte avant démarr
json	Bibliothèque standard Python	<code>json.dumps(scene)</code> convertit le tableau Python en texte JSON. <code>json.loads(texte)</code>
deque	collections (standard)	File d'attente double. Utilise dans le BFS. <code>deque.append()</code> ajoute à droite, <code>deque</code>

3.2 Imports de serveur_nao.py (Python 2.7)

Import	Provenance	Utilisation dans le code
socket	Bibliothèque standard	Crée le serveur TCP. <code>socket()</code> , <code>setsockopt()</code> , <code>bind()</code> , <code>listen()</code> , <code>accept()</code> , <code>recv()</code> , <code>send</code>
sys	Bibliothèque standard	<code>sys.stdout.flush()</code> force l'affichage immédiat des print dans les logs Choregraphe
math	Bibliothèque standard	<code>math.pi</code> vaut 3.14159... Utilise pour les rotations : <code>math.pi = 180</code> degrés, <code>math.pi/</code>
time	Bibliothèque standard	<code>time.sleep(2)</code> au démarrage pour laisser le robot virtuel s'initialiser. <code>time.sleep(1)</code>
select	Bibliothèque standard	<code>select.select([socket],[],[],1.0)</code> écoute le socket avec un timeout d'1 seconde. Per
json	Bibliothèque standard	<code>json.loads()</code> reconvertit le JSON reçu en tableau Python. Identique au client mais
ALProxy	naoqi (SDK Aldebaran)	Interface avec les modules du NAO virtuel. <code>ALProxy('ALMotion')</code> donne accès au

4. serveur_nao.py — Ligne par ligne

4.1 Variables de configuration

```
IP = "127.0.0.1"      # adresse locale = la meme machine
PORT_NAO = 9559       # port de l'API NAOqi (fixe par Aldebaran)
PORT_SERVER = 9561    # port sur lequel notre serveur ecoute
```

4.2 Fonction log

```
def log(msg):
    print(msg)          # affiche dans les logs Choregraphe
    sys.stdout.flush()  # force l'affichage immediat
```

Sans flush(), Choregraphe bufferise les print et les affiche tous en meme temps a la fin, ce qui rend le debug impossible. flush() vide le buffer immediatement.

4.3 Fonction verifier_case

```
def verifier_case(x, y):
    if scene is None:                # scene pas encore recue
        return "OBSTACLE"
    if not (0 <= y < len(scene)      # y dans les limites
            and 0 <= x < len(scene[0])): # x dans les limites
        return "OBSTACLE"          # hors de la grille
    return "OBSTACLE" if scene[y][x] in ("X", "0") else "OK"
    # X=mur, 0=inconnu -> obstacle
    # tout le reste (1, o, objectif, N, E) -> accessible
```

4.4 Initialisation des proxies NAO

```
time.sleep(2) # attend que le robot virtuel démarre complètement

motion = ALProxy("ALMotion", IP, PORT_NAO)
# donne acces aux commandes de deplacement physique du NAO
# moveTo(x, y, theta) : avance/recule/tourne

posture = ALProxy("ALRobotPosture", IP, PORT_NAO)
# donne acces aux postures predefinies
# goToPosture("StandInit", 0.5) met le robot debout en 0.5 secondes

life = ALProxy("ALAutonomousLife", IP, PORT_NAO)
# gere la vie autonome du NAO
# setState("disabled") coupe les animations idle et le dialogue auto
```

Pourquoi time.sleep(2) ? Quand Choregraphe lance le script, le robot virtuel n'est pas encore totalement initialise. Sans ce delai, ALProxy echoue avec une erreur 'socket not connected'.

4.5 Creation du serveur socket

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# AF_INET = protocole IPv4 (adresses type 192.168.x.x ou 127.0.0.1)
# SOCK_STREAM = TCP : connexion fiable, ordonnee, sans perte

server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```

# SOL_SOCKET = niveau socket (pas TCP ni IP)
# SO_REUSEADDR = autorise la reutilisation du port immediatement
# sans ca : erreur "address already in use" si on relance vite

server.bind(("0.0.0.0", PORT_SERVER))
# bind = associe le socket a une adresse et un port
# "0.0.0.0" = ecoute sur TOUTES les interfaces reseau
# (loopback 127.0.0.1, WiFi, Ethernet...)

server.listen(1)
# met le socket en mode ecoute passive
# 1 = taille de la file d'attente de connexions entrantes

```

4.6 Attente de connexion avec select

```

while True:
    if select.select([server], [], [], 1.0)[0]:
        # select.select(lecture, ecriture, erreur, timeout)
        # [server] = liste des sockets a surveiller en lecture
        # []      = pas de surveillance en ecriture
        # []      = pas de surveillance d'erreur
        # 1.0     = timeout de 1 seconde
        # retourne ([sockets prêts], [], [])
        # [0] = premiere liste = sockets prêts a lire
        # si non vide : une connexion arrive
        conn, addr = server.accept()
        # accept() accepte la connexion
        # conn = socket de communication avec CE client
        # addr = (ip, port) du client
        break

```

Pourquoi select et pas juste accept() ? accept() sans select bloque le thread Python indefiniment. Choregraphe ne peut pas interrompre un thread bloque. Avec select et timeout=1.0, le script verifie toutes les secondes si Choregraphe veut l'arreter, ce qui permet au bouton stop de fonctionner.

4.7 Reception de la scene JSON

```

buffer_scene = ""
while "\n" not in buffer_scene:
    # lit jusqu'a recevoir le marqueur de fin de message
    buffer_scene += conn.recv(4096).decode()
    # recv(4096) = lit jusqu'a 4096 bytes du reseau
    # decode() = convertit bytes -> texte Python

scene_json, reste = buffer_scene.split("\n", 1)
# split("\n", 1) = coupe au premier \n, en 2 parties max
# scene_json = le JSON de la scene
# reste = ce qui a pu arriver juste apres (commandes anticipees)

scene = json.loads(scene_json)
# json.loads = convertit texte JSON -> objet Python
# scene redevient une liste de listes identique a celle du client

```

4.8 Initialisation du NAO

```

try:
    conn.send("INIT:Activation des moteurs...\n".encode())
    # encode() = convertit texte -> bytes (obligatoire pour socket)
    # "INIT:" = prefixe reconnu par le client pour afficher en [INIT]

```

```

    life.setState("disabled")
    # coupe la vie autonome : animations, dialogue, awareness
except Exception as e:
    # si ALAutonomousLife indisponible, on continue quand meme
    log("ALAutonomousLife non disponible : {}".format(e))

conn.send("INIT:WakeUp...\n".encode())
motion.wakeUp()
# active les moteurs du robot (sort du mode rest)
# sans wakeUp(), moveTo() est ignore

conn.send("INIT:Mise en position...\n".encode())
posture.goToPosture("StandInit", 0.5)
# met le robot en position debout standard
# 0.5 = vitesse (0=lent, 1=rapide)

conn.send("PRET\n".encode())
# signal au client : NAO est pret, tu peux envoyer les commandes

```

4.9 Dictionnaire des mouvements

```

MOUVEMENTS = {
    "rotation": (
        lambda: motion.moveTo(0, 0, math.pi),
        # moveTo(x=0, y=0, theta=pi) = rotation pure 180 degrees
        # math.pi = 3.14159 radians = 180 degrees
        "NAO a effectue la rotation 180\n"
    ),
    "rotation_gauche": (
        lambda: motion.moveTo(0.001, 0, math.pi / 2),
        # 0.001m de micro-translation pour forcer l'execution
        # math.pi/2 = 1.5708 radians = 90 degrees a gauche
        "NAO a effectue la rotation gauche\n"
    ),
    "avant": (
        lambda: motion.moveTo(0.3, 0, 0),
        # avance de 0.3 metres = 30 cm
        "NAO a avance\n"
    ),
    "gauche": (
        lambda: (
            motion.moveTo(0.001, 0, math.pi / 2), # tourne 90 degrees
            time.sleep(1), # pause 1 seconde
            motion.moveTo(0.3, 0, 0) # avance 30 cm
        ),
        "NAO a tourne a gauche et avance\n"
    ),
}
# Utilisation :
action, feedback = MOUVEMENTS[commande]
action() # execute le mouvement
conn.send(feedback.encode()) # envoie confirmation au client

```

Pourquoi lambda ? Un dictionnaire ne peut stocker que des valeurs, pas des appels de fonction. La lambda permet de stocker la fonction sans l'exécuter immédiatement. action() l'exécute au moment voulu.

5. scene_idc.py — Ligne par ligne

5.1 La scene — structure et symboles

```
scene = [  
    ["X","X","X","X","X","X","X"],    # ligne y=0  
    ["E","N","o","o","1","1","X"],    # ligne y=1  
    ["X","o","X","o","o","o","X"],    # ligne y=2  
    ...  
]  
# scene[y][x] = case a la colonne x, ligne y  
# ATTENTION : y d'abord, x ensuite (inverse de la convention habituelle)  
# scene[1][0] = "E" (colonne 0, ligne 1)  
# scene[1][1] = "N" (colonne 1, ligne 1)
```

Symbole	Signification	Role dans le code
X	Mur infranchissable	OBSTACLE dans verifier_case cote serveur
0	Case inconnue	OBSTACLE dans verifier_case
1	Case accessible	OK mais hors itineraire, non explore par BFS
N	Position initiale NAO	trouver_position('N') -> coordonnees de depart
E	Direction de regard	Calcul orientation initiale du robot
o (cercle)	Itineraire dessine	Seules cases explorees par le BFS
objectif	Destination finale	But du BFS, fin du chemin

5.2 trouver_position — comment le NAO connait sa position

```
def trouver_position(symbole):  
    for y in range(len(scene)):  
        for x in range(len(scene[0])):  
            if scene[y][x] == symbole:  
                return [x, y]  
    return None  
  
NAO = trouver_position("N")  
# NAO vaut par exemple [1, 1]  
# NAO[0] = x = colonne  
# NAO[1] = y = ligne
```

Comment la position du NAO est maintenue ? NAO est une liste Python [x, y]. Elle est initialisee par trouver_position('N') au demarrage. Apres chaque mouvement reussi, calculer_deplacement() retourne (dx, dy) et on fait NAO[0] += dx et NAO[1] += dy. Comme NAO est une liste (mutable), la modification est visible partout dans le code sans avoir besoin de la retourner ou de la declarer global.

```
# Exemple de suivi de position :  
NAO = [1, 1]                                # position initiale N  
  
# Apres commande "avant" (orientation=2, face au bas) :  
# calculer_deplacement("avant") retourne (0, 1)  
NAO[0] += 0  # x inchange  
NAO[1] += 1  # y passe de 1 a 2  
# NAO = [1, 2] -> le robot est maintenant en ligne 2
```



```

# Apres commande "gauche" (tourne + avance vers droite) :
# calculer_deplacement("gauche") : orientation passe de 2 a 1
# DEPLACEMENT[1] = (1, 0)
NAO[0] += 1 # x passe de 1 a 2
NAO[1] += 0 # y inchange
# NAO = [2, 2]

```

5.3 calculer_orientation_initiale

```

def calculer_orientation_initiale(nao, e):
    dx, dy = e[0] - nao[0], e[1] - nao[1]
    # dx, dy = vecteur de N vers E
    # Si E=[0,1] et N=[1,1] : dx=0-1=-1, dy=1-1=0
    return {
        (0, -1): 0, # E au dessus de N -> face au haut
        (1, 0): 1, # E a droite de N -> face a droite
        (0, 1): 2, # E en dessous de N-> face au bas
        (-1, 0): 3 # E a gauche de N -> face a gauche
    }.get((dx, dy), 0)
    # .get(cle, valeur_defaut) retourne 0 si (dx,dy) inconnu

```

Les 4 orientations : 0=haut (y decroit), 1=droite (x croit), 2=bas (y croit), 3=gauche (x decroit). Cette convention est cruciale pour toute la logique de deplacement.

5.4 Le BFS — suivre_chemin_scene

```

def suivre_chemin_scene():
    start = (NAO[0], NAO[1])
    # position de depart = tuple (pas liste car utilise dans un set)

    but = tuple(trouver_position("objectif"))
    # position de l'objectif = tuple

    queue = deque([(start, [start])])
    # file d'attente initialisee avec le point de depart
    # chaque element = (case_courante, chemin_parcouru_jusqua_la)

    visite = {start}
    # ensemble des cases deja explorees
    # set() pour verifier l'appartenance en O(1) au lieu de O(n)

    while queue:
        (x, y), chemin_actuel = queue.popleft()
        # retire le premier element (FIFO = First In First Out)
        # decompacte en : case courante + chemin jusqu'ici

        if (x, y) == but:
            return [[p[0], p[1]] for p in chemin_actuel]
            # objectif atteint : convertit tuples en listes et retourne

        for dx, dy in [(0,1),(0,-1),(1,0),(-1,0)]:
            # explore les 4 directions : bas, haut, droite, gauche
            nx, ny = x + dx, y + dy
            # coordonnees du voisin

            if (nx, ny) not in visite # pas deja explore
            and 0 <= ny < len(scene) # y dans les bornes
            and 0 <= nx < len(scene[0]): # x dans les bornes
                if scene[ny][nx] in ("o","objectif"):

```

```

        # case accessible pour l'itineraire
        visite.add((nx, ny))          # marque comme visite
        queue.append(                 # ajoute a la file
            ((nx, ny), chemin_actuel + [(nx, ny)])
        )

    return [list(start)] # aucun chemin trouve : retourne juste le depart

```

5.5 deduire_commandes — de positions a commandes

```

def deduire_commandes(positions, orientation_initiale):
    DIR = {(0,-1): 0, (1,0): 1, (0,1): 2, (-1,0): 3}
    # dictionnaire : vecteur de deplacement -> orientation

    commandes = []
    orientation = orientation_initiale # orientation courante du robot

    for i in range(1, len(positions)):
        px, py = positions[i-1] # case precedente
        nx, ny = positions[i]   # case suivante
        dx, dy = nx-px, ny-py   # vecteur de deplacement

        direction_cible = DIR[(dx, dy)]
        # direction vers laquelle il faut aller

        diff = (direction_cible - orientation) % 4
        # difference d'angle (modulo 4 pour gerer les cas circulaires)
        # diff=0 : deja en face -> "avant"
        # diff=1 : tourner 90 deg droite -> "droite"
        # diff=3 : tourner 90 deg gauche -> "gauche"
        # diff=2 : demi-tour -> "rotation" + "avant"

        if diff == 0:
            commandes.append("avant")
        elif diff == 1:
            commandes.append("droite")
            orientation = (orientation + 1) % 4 # met a jour orientation
        elif diff == 3:
            commandes.append("gauche")
            orientation = (orientation - 1) % 4
        elif diff == 2:
            commandes.append("rotation")
            orientation = (orientation + 2) % 4
            commandes.append("avant") # apres demi-tour, avancer

    return commandes

```

5.6 Les Events de synchronisation

```

arret = threading.Event()
# .set() -> signale que l'arret est demande
# .is_set()-> True si arret demande
# utilise par écouter_arret() et verifie dans la boucle principale

feedback_recu = threading.Event()
# .clear() -> remet a zero (avant d'envoyer une commande)
# .set() -> signale qu'une reponse est arrivee
# .wait() -> bloque jusqu'a ce que .set() soit appele
# utilise pour attendre la reponse du serveur

```

```

obstacle_detecte = threading.Event()
# .set()    -> signale qu'un OBSTACLE a ete reçu
# .clear()  -> remet a zero quand on recoit OK
# .is_set()-> True si obstacle detecte

```

Comment les Events synchronisent les threads ? Le thread principal envoie une commande puis appelle `feedback_recu.wait()`. Ça bloque le thread principal. Pendant ce temps, le thread `recevoir()` lit la réponse du serveur. Quand il reçoit quelque chose, il appelle `feedback_recu.set()` ce qui debloque le thread principal. Sans ça, le thread principal enverrait toutes les commandes d'un coup sans attendre que le NAO finisse chaque mouvement.

5.7 Les threads daemon

```

threading.Thread(target=ecouter_arret, daemon=True).start()
threading.Thread(target=recevoir, daemon=True).start()
# daemon=True = le thread s'arrete automatiquement quand
# le programme principal se termine
# sans daemon=True, le programme resterait bloqué
# meme apres la fin de la boucle principale

```

5.8 calculer_deplacement — mise a jour orientation et position

```

DEPLACEMENT = {
    0: (0, -1),    # face au haut : x inchangé, y diminue
    1: (1,  0),    # face a droite: x augmente, y inchangé
    2: (0,  1),    # face au bas  : x inchangé, y augmente
    3: (-1, 0)     # face a gauche: x diminue, y inchangé
}

def calculer_deplacement(cmd):
    global orientation # modifie la variable globale orientation
    if cmd == "rotation":
        orientation = (orientation + 2) % 4 # demi-tour
        return (0, 0) # pas de déplacement
    elif cmd == "avant":
        return DEPLACEMENT[orientation] # avance dans la direction actuelle
    elif cmd == "gauche":
        orientation = (orientation - 1) % 4 # tourne a gauche
        return DEPLACEMENT[orientation]    # avance dans la nouvelle direction
    ...

```

Ordre important dans gauche/droite : orientation est modifiée AVANT de lire `DEPLACEMENT[orientation]`. C'est voulu : gauche = tourne + avance, donc le déplacement sur la carte correspond à la nouvelle direction après rotation.

5.9 calculer_case_cible — sans modifier l'etat

```

def calculer_case_cible(cmd):
    o = orientation # copie locale, ne modifie PAS orientation globale
    if cmd == "avant":
        dx, dy = DEPLACEMENT[o]
    elif cmd == "gauche":
        dx, dy = DEPLACEMENT[(o - 1) % 4] # orientation apres rotation gauche
    elif cmd == "droite":
        dx, dy = DEPLACEMENT[(o + 1) % 4]
    else:
        return (NAO[0], NAO[1]) # pas de déplacement
    return (NAO[0] + dx, NAO[1] + dy)
# retourne les coordonnees de la case cible

```

SANS modifier orientation ni NAO

Pourquoi deux fonctions separees ? calculer_deplacement() modifie l'etat global (orientation). Elle est appelee apres confirmation du mouvement. calculer_case_cible() ne modifie rien. Elle est appelee pour la verification AVANT le mouvement suivant. Si on utilisait calculer_deplacement() pour la verification, l'orientation serait decalee et tout le reste serait faux.

6. La boucle d'execution principale

```
for i, commande in enumerate(chemin):
    # enumerate() donne l'index i ET la valeur commande
    # i va de 0 a len(chemin)-1

    if arret.is_set():
        break # ENTREE pressee -> on sort

    # ETAPE 1 : envoyer la commande
    feedback_recu.clear()      # remet l'event a zero
    envoyer(commande)          # envoie au serveur
    feedback_recu.wait()       # bloque jusqu'a reponse

    if arret.is_set():
        break # arret demande pendant l'attente

    # ETAPE 2 : mettre a jour la carte
    dx, dy = calculer_deplacement(commande)
    # met a jour orientation ET retourne le deplacement
    NAO[0] += dx
    NAO[1] += dy

    # ETAPE 3 : enregistrer et afficher
    historique_commandes.append(commande)
    historique.append("[{}/{}] {}".format(i+1, len(chemin), commande))
    afficher_scene()

    # ETAPE 4 : verifier la PROCHAINE case (si pas derniere commande)
    if i + 1 < len(chemin) and not verifier_prochaine_case(chemin[i+1]):
        retour_position_initiale()
        echec = True
        break
```

Pourquoi verifier apres et non avant ? Le robot avance D'ABORD, PUIS verifie ce qui est devant lui. Cela simule un vrai robot qui utilise ses capteurs depuis sa nouvelle position. La verification avant chaque commande (sauf la premiere) permet de detecter un obstacle avant de s'y cogner.

7. Retour en cas d'obstacle

```
INVERSES = {
    "rotation": "rotation",      # demi-tour inverse = demi-tour
    "avant":    "arriere",       # reculer annule avancer
    "arriere":  "avant",
    "gauche":   ("arriere", "rotation_droite"),
    # gauche = rotation gauche + avance
    # pour annuler : reculer D'ABORD, puis rotation droite
    # (l'ordre est inverse ET les actions sont inversees)
    "droite":   ("arriere", "rotation_gauche"),
}

def retour_position_initiale():
    for cmd in reversed(historique_commandes):
```

```
# reversed() = parcourt la liste de la fin vers le debut
inverse = INVERSES[cmd]
if isinstance(inverse, tuple):
    # tuple = plusieurs commandes a executer
    for c in inverse:
        executer_commande(c)
else:
    # string = une seule commande
    executer_commande(inverse)
```

Exemple concret : chemin parcouru = [rotation, avant, avant, gauche]. Pour revenir : reversed = [gauche, avant, avant, rotation]. gauche inverse = (arriere, rotation_droite). avant inverse = arriere. rotation inverse = rotation. Commandes executees : arriere, rotation_droite, arriere, arriere, rotation.

8. Affichage de la scene

```
cases_itineraire = {(p[0], p[1]) for p in positions_chemin[1:]}
# set comprehension : cree un ensemble de tuples (x,y)
# positions_chemin[1:] = toutes les positions sauf la position de depart N
# (N est deja affiche avec un symbole different)

cases_non_utilisees = {
    (x, y)
    for y in range(len(scene))
    for x in range(len(scene[0]))
    if scene[y][x] == "o" and (x, y) not in cases_itineraire
}
# cases qui sont des "o" dans la scene mais pas dans le chemin optimal
# = branches ignorees par le BFS

SYMBLES_AFFICHAGE = {
    "X": " X ", "1": " 1 ", "0": " 0 ",
    "E": " E ", "N": " triangle ", "objectif": " objectif "
}
# dictionnaire pour eviter les if/elif repetitifs
# .get(cle, valeur_defaut) retourne " " si symbole inconnu

def afficher_scene():
    os.system("cls" if os.name == "nt" else "clear")
    # cls = efface le terminal Windows
    # clear = efface le terminal Linux/Mac
    # os.name == "nt" = True sur Windows
    for y in range(len(scene)):
        for x in range(len(scene[0])):
            if [x, y] == NAO:          # 1. Robot (priorite max)
                ligne += " diamant "
            elif scene[y][x] == "objectif": # 2. Objectif
                ligne += " objectif "
            elif (x,y) in cases_itineraire: # 3. Chemin optimal
                ligne += " o "
            elif (x,y) in cases_non_utilisees: # 4. Chemin non utilise
                ligne += " . "
            else:                        # 5. Symboles normaux
                ligne += SYMBLES_AFFICHAGE.get(scene[y][x], " ")
```

Ordre de priorite important : l'objectif est verifie AVANT cases_itineraire car il fait partie du chemin BFS (derniere case). Sans cette priorite, l'objectif serait masque par le symbole 'o'.

9. Flux d'execution complet

Etape	Client (scene_idc.py)	Serveur (serveur_nao.py)
1	Calcule BFS + commandes	—
2	Verifie chemin non vide	—
3	connect(127.0.0.1:9561)	select() -> accept()
4	send(scene JSON + \n)	recv() -> json.loads()
5	Attend INIT messages	life.setState + wakeUp + StandInit
6	Attend 'PRET'	send('PRET\n')
7	Decompte 5 secondes	select() boucle en attente
8	feedback_recu.clear()	—
9	send('avant\n')	recv() -> moveTo(0.3,0,0)
10	feedback_recu.wait()	send('NAO a avance\n')
11	recevoir() : feedback_recu.set()	—
12	calculer_deplacement() -> NAO mis a jour	—
13	send('verifier:[x,y]\n')	recv() -> verifier_case(x,y)
14	feedback_recu.wait()	send('OK\n') ou send('OBSTACLE\n')
15 OK	Continue avec commande suivante	—
15 OBS	retour_position_initiale()	Execute commandes inverses
16	send('stop\n')	stopper=True -> fermeture